



Bloc 1 : Construction et alimentation d'une infrastructure de gestion de données

Où planifiez vos prochaines vacances?

[Dépôt Github](#)



Contexte et besoins du projet

Le besoin chez Kayak

- Développer une application de recommandation de destinations de vacances basée sur des données réelles afin d'aider les voyageurs à mieux choisir leur prochain séjour
 - **70% de leurs clients le demandent**
- Pour identifier et recommander les meilleures destinations et hébergements à un instant donné, Kayak demande d'utiliser des informations réelles
 - la météo
 - les meilleurs hôtels disponibles

Objectifs et périmètre du projet

Objectifs

- Collecter des données pour chacune des 35 villes destinations :
 - données météorologiques avec OpenWeatherMap (OWM) : prévisions à 4j
 - données sur les hôtels les mieux notés sur Booking.com
- Stocker ces données
 - dans un **data lake**
 - les **extraire, transformer** et les **charger (ETL)**
 - vers un **data warehouse**

Afin de disposer de données exploitables pour l'application de recommandation

Architecture de la solution proposée

Un pipeline automatisé en 7 étapes

1. Géocodage Nominatim : coordonnées géo (long., lat.) des 35 villes choisies en config
 2. API OpenWeatherMap : prévisions de J+1 à J+4 (free tier 1000 req/j)
 3. Score météo pondéré (0-100 points): permet le classement des villes destination
 4. Scraping de [Booking.com](https://www.booking.com) (avec Playwright) : les 20 premiers hôtels des 5 meilleurs destinations
 5. Upload vers le data lake qui sera un bucket AWS S3
 6. Chargement des données vers un data warehouse : BdD PostgreSQL (Neon)
 7. Dashboard de visualisation interactive (Streamlit)
- **Pipeline est exécuté automatiquement tous les jours à 6h00**
 - **Purge locale des fichiers effectuée pour météo et hôtels (pour les X dates précédents ou X jours précédents, X configurable)**

Choix des données (1)

Nominatim

- longitude, latitude
- le nom de ville tel que le propose Nominatim (utile pour former l'adresse de l'hôtel)

OpenWeatherMap

- date du jour et heure (8 créneaux de 3h)
- température, température ressentie, temp. min et max en degrés
- pression atmosphérique (mmHg)
- % humidité, % couverture nuageuse, vitesse du vent (km/h)
- probabilité d'avoir de la pluie (%)

Booking.com

- nom de l'hôtel, lat.+long., description, note, URL booking, address, ville, code postal

Choix des données (2) : modèle de scoring météo

Scoring fondé sur le Tourism Climatic Index de Mieczkowski :

- **score composite 0–100** fondé sur les 6 critères météo retenus (créneaux 08h–20h)
- **classification par tranche** des valeurs de météo retenues. Exemples :
 - temp. $<0^{\circ}$ = très froid $\Rightarrow$ score 0/10, de 18 à 28° = idéal $\Rightarrow$ 10/10, $>33^{\circ}$ $\Rightarrow$ 0/10
 - proba pluie : score = $(1 - \text{rain_proba}) \times 100$, etc.
- pour chaque jour de prévision : moyenne des valeurs de 8-20h puis **score pondéré**
 - Température ressentie 30%
 - Probabilité de pluie 30%
 - Vitesse du vent 15%
 - Couverture nuageuse 10%
 - Humidité 10%
 - Pression atmosphérique 5%
- **score final = moyenne des 4j + pénalité de variabilité météo** : -10 pts si min ou max dépasse $\text{mean} \pm 3\sigma$ (jusqu'à -20 pts au total)

Pipeline data : robustesse

- Conçu pour s'exécuter de façon **autonome en production**
 - CRON qui s'exécute à 6h00
 - purge locale fichiers météo et hôtels des X jours précédents ou à moins X jours
- **Résilience**
 - **cache JSON local** pour les localisations des villes et résultats météo: pas d'appels redondants et évite d'envoyer des requêtes inutiles qui bloquerait l'accès à Nominatim ou OWM
 - **mécanismes de réessai** avec backoff exponentiel et jitter sur les erreurs de requêtage (ex: 429, 5xx) vers Nominatim ou OWM
 - **idempotence** : on vérifie les fichiers existants avant l'appel à l'API
- **Organisation et historisation**
 - **répertoires de la forme 'yyyyMMdd/'** en local ou S3 + file naming patterns
 - chaque exécution est indépendante de la précédente

Pipeline data : coûts de la solution

- solution AWS RDS non retenue par rapport à Neon
- déploiement sur cloud (AWS EC2, GCP, OVH?) à prévoir pour le pipeline

Source	Limites	Plan	Coûts
Nominatim	1 req / sec max	Gratuit	0€
OpenWeatherMap	1000 req/j max	Free tier	0€
AWS S3	5Go	pay-as-you-go	0€
Neon PostgreSQL	5 bdd	Serverless plan	0€
AWS RDS	-	pay-as-you-go	>0€
Cloud server		OVH?	>0€

Architecture détaillée de l'ETL

EXTRACT

NOMINATIM : mise en cache
json/cities/city-{city_id}.json

OWM : mise en cache
weather/YYYYMMDD/weather-{city_id}-YYYYMMDD.json

Booking : mise en cache de
html/{city_id}_liste.html (pour
debug)

TRANSFORM

NOMINATIM : compilation des villes
dans csv/cities.csv

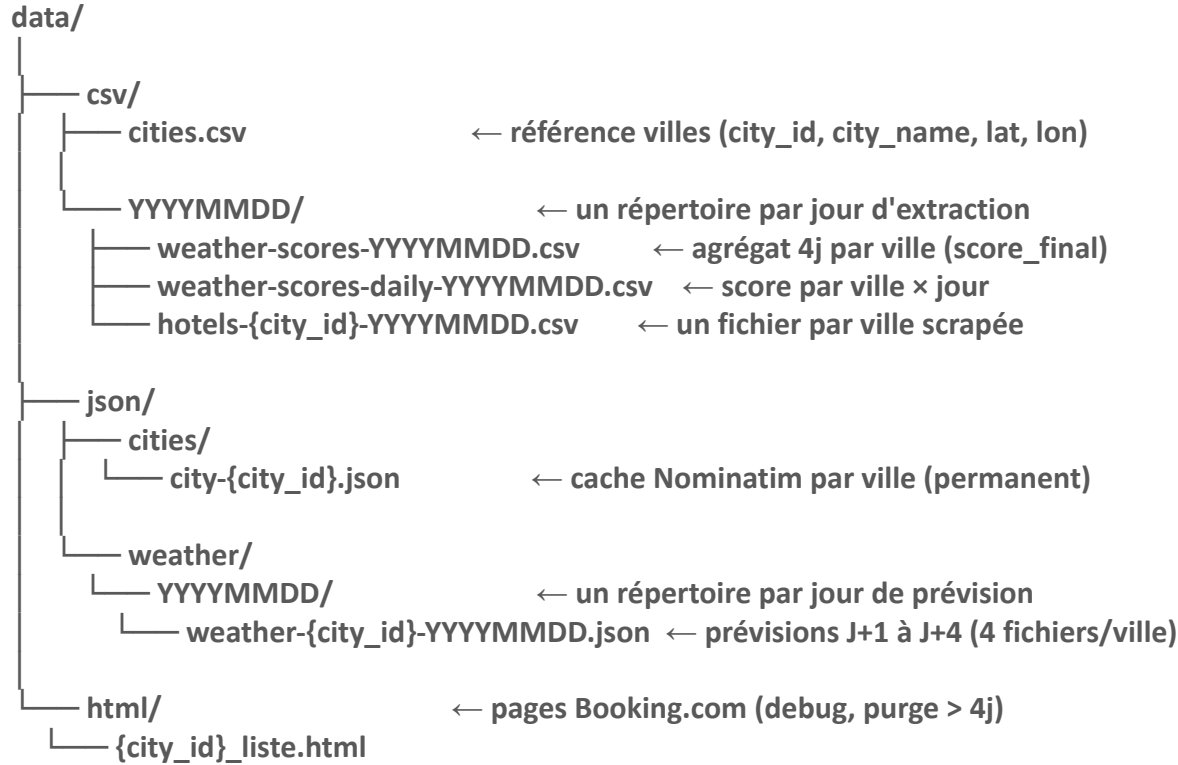
OWM :
csv/YYYYMMDD/weather-scores-YY
YMMDD.csv
csv/YYYYMMDD/weather-scores-dai
ly-YYYYMMDD.csv

Booking : en même temps que
l'extract
csv/YYYYMMDD/hotels-{city_id}-YYY
YMMDD.csv

LOAD

- upload vers S3/{PREFIX}
 - cities.csv
 - fichiers météo et hôtels
du jour dans YYYYMMDD/
- stratégies d'upload BDD
 - replace pour cities
 - replace pour les scores
météo
 - upsert pour les scores
météo par jour
 - merge + replace par plus
récent pour les hôtels

Arborescence locale de stockage



Arborescence data lake S3

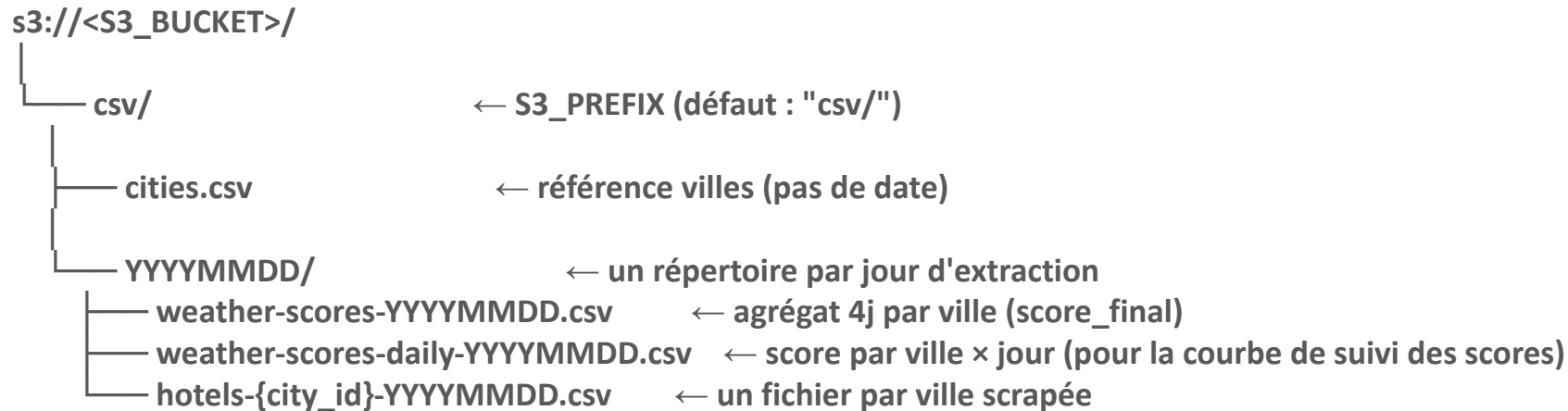


Schéma du data warehouse

5 tables PostgreSQL :

- **history** : contient la date de chargement de chacune des tables
- **cities** : contient les données Nominatim
- **weather_scores** : les scores calculés pour le jour J
- **weather_scores_daily** : pour avoir un historique des scores détaillés
- **hotels** : les données relatives des hôtels

Dashboard en production (1)

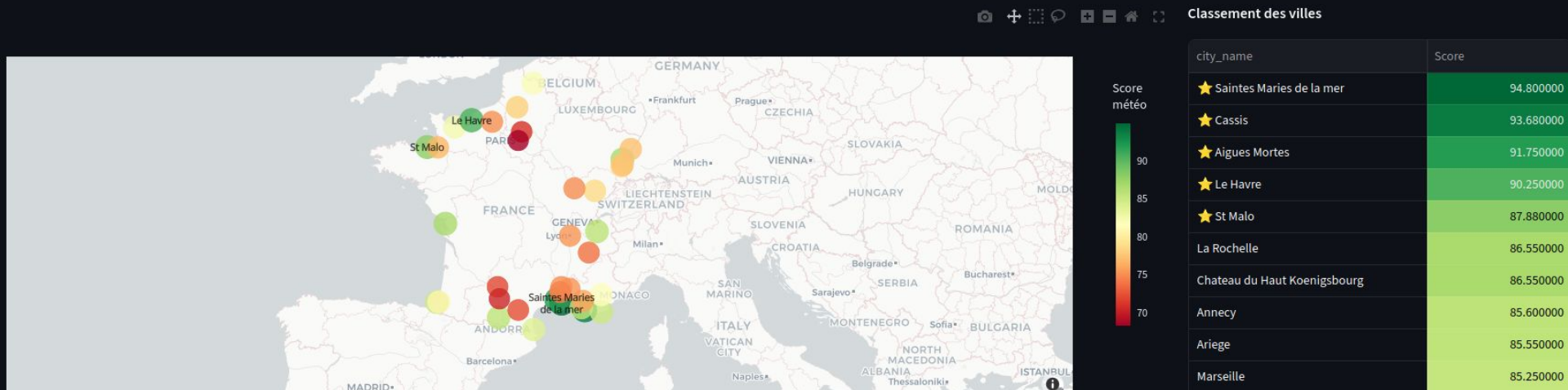
<https://jedha-project-02-kayak-4kzhtcxgijwncdslldqjdmq.streamlit.app/>

Carte interactive des destinations avec les 5 premiers dans le tableau à droite

KAYAK — Top destinations météo en France

Prévisions météo du 21 June 2026 au 24 June 2026 — données extraites le 20 June 2026

Destinations météo — scores sur 4 jours (36 villes)



Dashboard en production (2)

Carte interactive des hôtels (et popup) avec tableau à droite pour les infos

Hôtels à Cassis

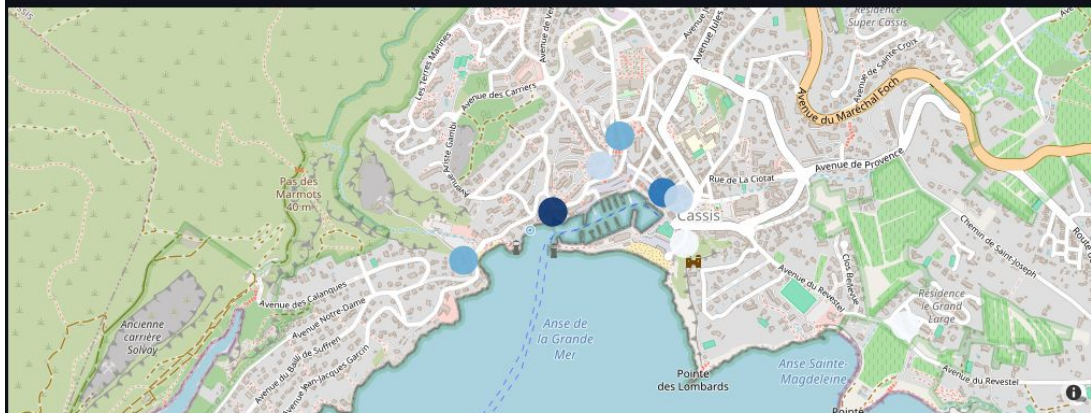


Tableau 2 — Top 10 hôtels

Hôtel	Score	Adresse
L'Aiguade, appartement d'exception	9.6	12 Avenue de l'Amiral Ganteaume, 13260 Cassis
La Martine	9.5	2 Rue Lamartine, 13260 Cassis
AU BORD DE L'EAU Studio climatisé	9.4	Le calendal 2 Avenue des Calanques, 13260 Cassis
Villa Bobnell	9.4	4 Impasse du 8 mai, 13260 Cassis
Magnifique Appartement à Cassis	9.4	19 Avenue de la Viguerie, 13260 Cassis
Nistoun par Dodo à Cassis	9.3	2 ème étage 1 Rue Dr Séverin Icard, 13260 Cassis
SunSet Cassis	9.3	3 Boulevard Jean Jaurès, 13260 Cassis
Appart Bianca T2 - Terrasse - Park	9.3	40 Avenue Jean Jacques Garcin, 13260 Cassis
Le Grand Large par Dodo à Cassis	9.2	595 avenue du revestel Résidence Le Grand Large
SAM Cassis	9.2	13b Rue Alexandre Rossat, 13260 Cassis

Difficultés ou limitations rencontrées

- **Nominatim** : limitation du nombre de requêtes -> cache + pause
- **OpenWeatherMap** : limitation à 4j + gestion d'erreurs API + choix des données
- **Modèle de scoring météo** : choix des données et des agrégations pour le modèle
- **Scraping booking.com** : un anti-bot agressif qui a nécessité une bascule de scrapy vers Playwright pour mieux simuler le comportement d'un humain (délais de clic, fermeture popups)
- **Construire et tester une Architecture autonome et résiliente** :
pas de Notebook -> des scripts indépendants, déploiement sous forme d'un pipeline avec exécution quotidienne

Améliorations possibles

- **Utiliser un orchestrateur de pipeline :**
 - **Airflow ou Jenkins**
- **Interface web pour téléphones mobiles**
 - **par exemple avec une stack Nextjs / Typescript / Bootstrap CSS**

Jedha - Data Science Fullstack

Certification CDSD (Concepteur développeur en science des données)



Questions ?